UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS

Programación 2

Justin alexander Antón Hernández 9941-25-20386

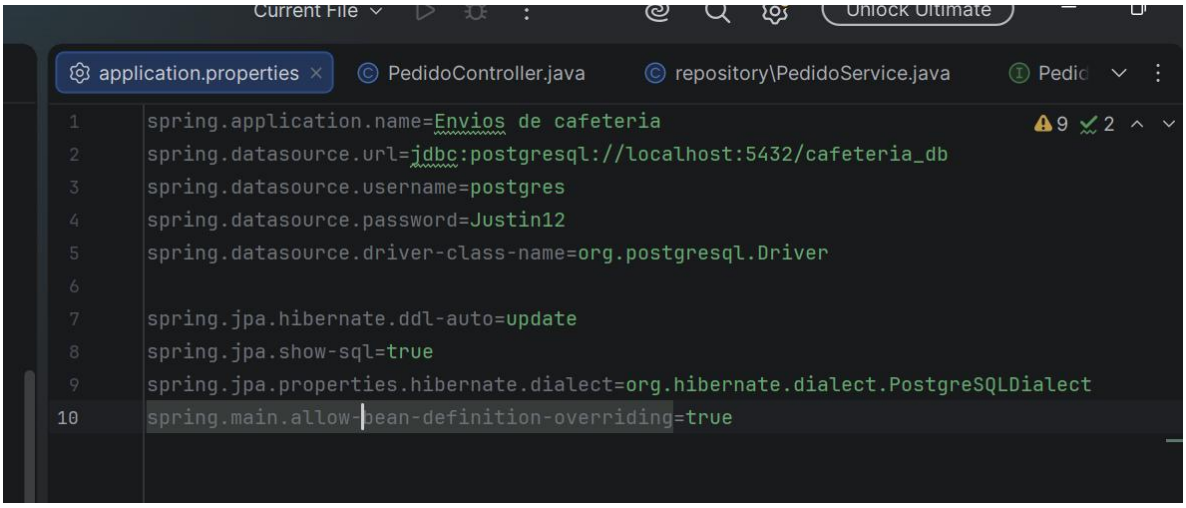
Guatemala 18 de octubre del 2026

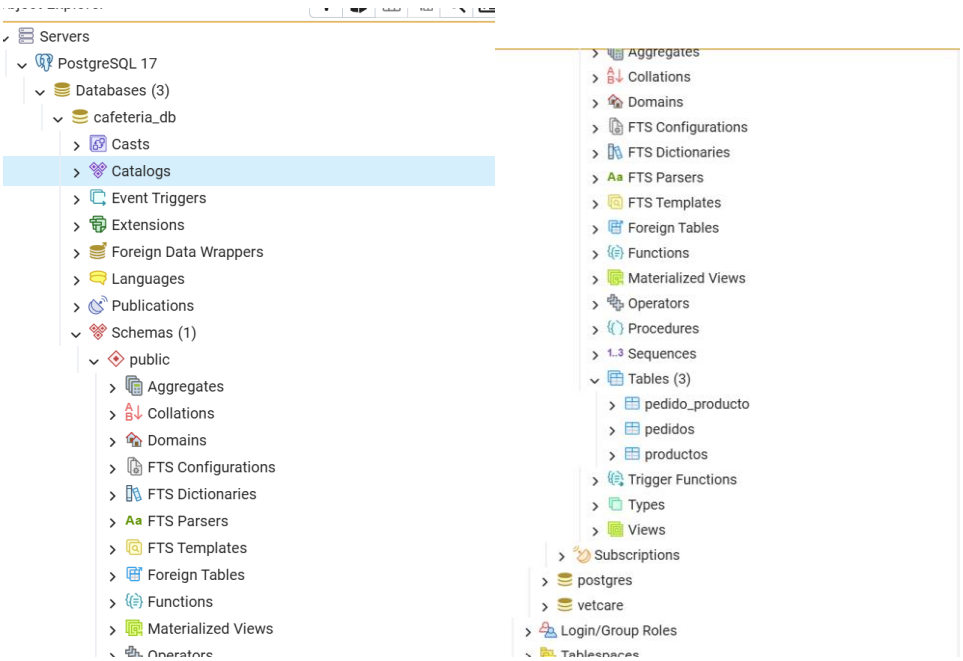
Avance 1

1. Definición del problema

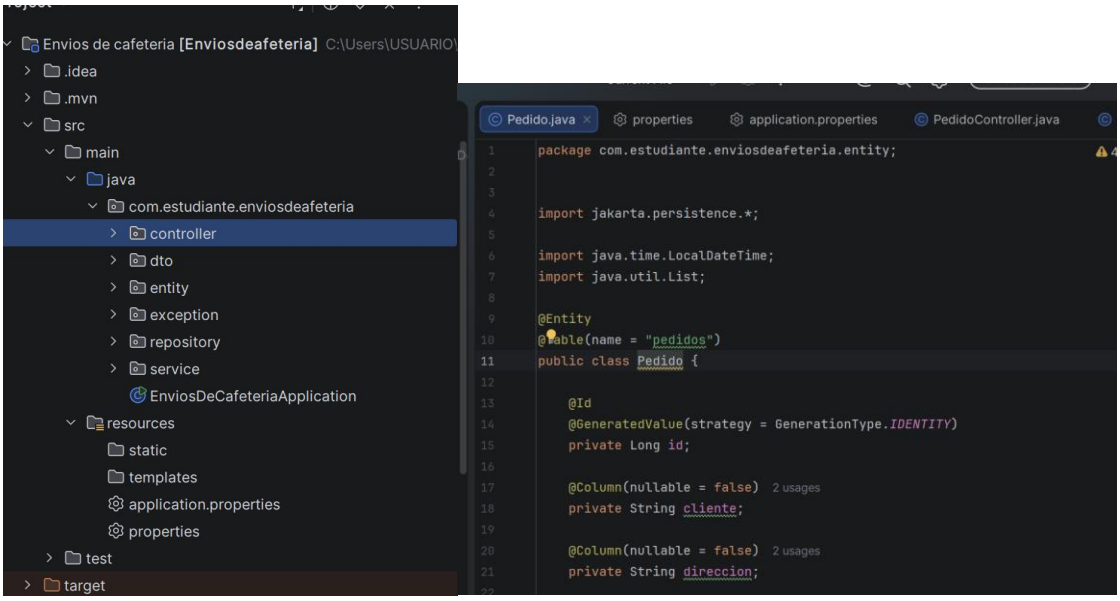
El proyecto consiste en un sistema web para administrar los envíos de una cafetería. Permite registrar los pedidos de los clientes, llevar el control de las direcciones de entrega, calcular los totales y gestionar los productos que componen cada pedido (como cafés o sándwiches), así como el estado en el que se encuentra la entrega.

Evidencia de la conexión con PostgreSQL y creación de tablas





Evidencia de la creación de Entidades y Relaciones


Envios de cafeteria [Enviosdeafeteria] C:\Users\USUARIO
.idea
.mvn
src
main
java
com.estudiante.enviosdeafeteria
controller
dto
entity
exception
repository
service
EnviosDeCafeteriaApplication
resources
static
templates
application.properties
properties
test
target
Pedido.java
properties
application.properties
PedidoController.java
package com.estudiante.enviosdeafeteria.entity;
import jakarta.persistence.*;
import java.time.LocalDateTime;
import java.util.List;
@Entity
@Table(name = "pedidos")
public class Pedido {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
@Column(nullable = false) 2 usages
private String cliente;
@Column(nullable = false) 2 usages
private String direccion;

```java
@Column(nullable = false)  2 usages
private Double total;

@Column(nullable = false)  2 usages
private String estado; // Ej: "PENDIENTE", "ENVIADO", "ENTREGADO"

@Column(nullable = false)  2 usages
private LocalDateTime fecha;

@ManyToMany  2 usages
@JoinTable(
        name = "pedido_producto",
        joinColumns = @JoinColumn(name = "pedido_id"),
        inverseJoinColumns = @JoinColumn(name = "producto_id")
)
private List<Producto> productos;

public Pedido() {}  2 usages

public Long getId() { return id; }  2 usages
public void setId(Long id) { this.id = id; }  no usages
```

```java
public class Pedido {  ⚠ 4  ✓ 25

    public String getCliente() { return cliente; }  2 usages
    public void setCliente(String cliente) { this.cliente = cliente; }  2 usages

    public String getDireccion() { return direccion; }  2 usages
    public void setDireccion(String direccion) { this.direccion = direccion; }  2 usages

    public Double getTotal() { return total; }  2 usages
    public void setTotal(Double total) { this.total = total; }  2 usages

    public String getEstado() { return estado; }  2 usages
    public void setEstado(String estado) { this.estado = estado; }  4 usages

    public LocalDateTime getFecha() { return fecha; }  2 usages
    public void setFecha(LocalDateTime fecha) { this.fecha = fecha; }  2 usages

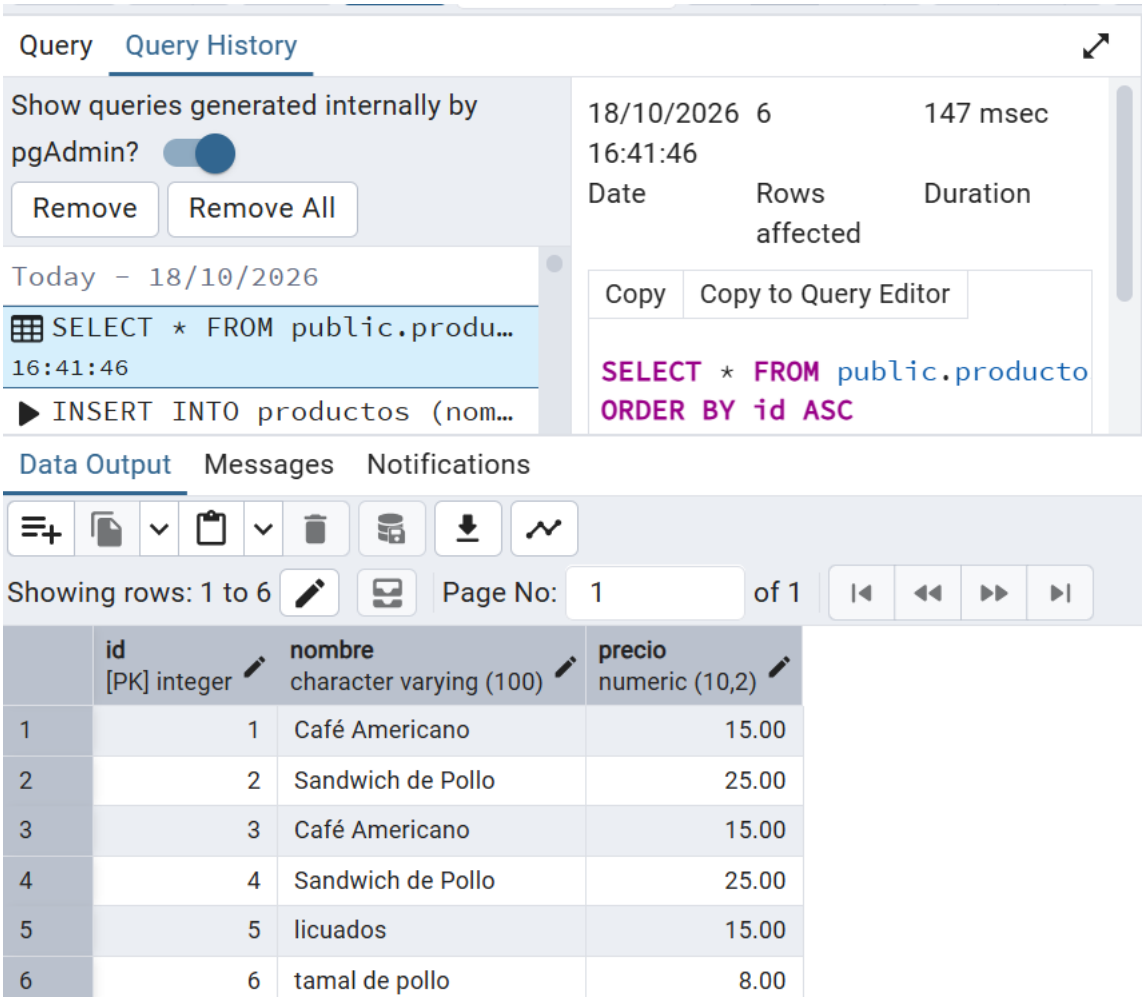
    public List<Producto> getProductos() { return productos; }  no usages
    public void setProductos(List<Producto> productos) { this.productos = productos; }
}
```

```java
package com.estudiante.enviosdeafeteria.repository;  ⚠ 1  ✓ 1

import com.estudiante.enviosdeafeteria.entity.Pedido;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository  5 usages
public interface PedidoRepository extends JpaRepository<Pedido, Long> {

    List<com.estudiante.enviosdeafeteria.repository.Pedido> findByEstado(String estado)
}
```



	id [PK] integer	nombre character varying (100)	precio numeric (10,2)
1	1	Café Americano	15.00
2	2	Sandwich de Pollo	25.00
3	3	Café Americano	15.00
4	4	Sandwich de Pollo	25.00
5	5	licuados	15.00
6	6	tamal de pollo	8.00

Reflexión final

Hasta este primer avance, he logrado configurar el entorno de desarrollo con Spring Boot y Maven, así como establecer la conexión con la base de datos PostgreSQL. El mayor reto fue comprender cómo Spring Data JPA maneja la creación de tablas automáticamente a través de las entidades y cómo se estructura una relación de Muchos a Muchos. Para la siguiente fase, planeo implementar la lógica de negocio (servicios) y los controladores REST para que el sistema sea totalmente funcional.